



UNIVERSIDAD AMERICANA

Facultad de Ingeniería y Arquitectura (FIA)

Asignatura: Álgebra Lineal (MTM0120)

ACTIVIDADES DE PROGRAMACIÓN EN PYTHON PROGRAMA 5

Módulo de Álgebra de Matrices: Operaciones, Determinantes e Inversa

Proyecto Integrador: Calculadora de Álgebra Lineal

(Módulo III: Álgebra de Matrices)

Modalidad

Grupal (Máximo 4 estudiantes) con defensa/exposición individual.

Segundo Corte Evaluativo

1. ¿De qué trata esta actividad?

Como futuros ingenieros, es fundamental comprender el funcionamiento interno de las operaciones matriciales antes de depender de librerías comerciales. En este programa van a construir un único módulo en Python que reúne todo el Módulo III de la Calculadora: las operaciones con matrices (suma, resta, multiplicación por un escalar, producto matricial y transposición), el cálculo de determinantes, la matriz inversa (por Gauss-Jordan y por la matriz adjunta) y un verificador de propiedades con el que el propio programa comprueba las igualdades vistas en clase. Antes de operar, el programa debe validar las condiciones de cada operación: dimensiones compatibles, matriz cuadrada y determinante distinto de cero.

Este módulo se integra a la Calculadora como `modulos/modulo_matrices.py` y será reutilizado en programas posteriores (por ejemplo, para verificar que $A \cdot A^{-1} = I$ o que $L \cdot U = A$), por eso debe diseñarse como una pieza reutilizable y fácil de leer.

2. Reglas del Juego (Técnicas)

- Cero librerías externas: No pueden importar NumPy, SciPy ni funciones de álgebra de math. Todo debe programarse usando código base de Python (listas anidadas, condicionales if/else, bucles for/while y funciones propias). Se permite el módulo estándar fractions para mostrar resultados exactos como fracciones (así la comprobación $A \cdot A^{-1} = I$ no depende de errores de redondeo).
- Código documentado con criterio (no con cantidad): cada archivo .py lleva una docstring de módulo y cada función una docstring breve; los comentarios de línea se reservan para explicar el porqué (decisiones matemáticas, validación de dimensiones), no para repetir lo que el código ya dice. Las reglas y un ejemplo están en la sección 5.
- Estructura modular (Proyecto Integrador): entregar el módulo como `modulos/modulo_matrices.py`, importado desde `main.py`, con el Logotipo ASCII del Módulo 3 y la opción 0. Ver Teoremas Clave del Módulo, tal como se definió en la Tarea 4.

- Cálculo separado de la interfaz: las funciones que operan (suma, resta, escalar, producto, transpuesta, determinante e inversa) no usan input() ni print(); reciben matrices y devuelven resultados. Así podrán reutilizarlas en los módulos siguientes.

3. Lo que debe hacer el programa (Requerimientos)

1. Entrada:

- Menú interactivo con las opciones: 1. Suma, 2. Resta, 3. Multiplicación por Escalar, 4. Producto Matricial, 5. Transposición, 6. Determinante, 7. Inversa por Gauss-Jordan, 8. Inversa por Matriz Adjunta, 9. Verificador de propiedades y 0. **Ver Teoremas Clave del Módulo.**
- Pedir las dimensiones ($m \times n$) de la(s) matriz/matrices a ingresar; para las opciones 6, 7, 8 y 9 las matrices deben ser cuadradas ($n \times n$).
- Solicitar la entrada de los elementos fila por fila para cada matriz.
- Validar cada dato: las dimensiones deben ser enteros positivos y los elementos números válidos; ante un dato incorrecto, el programa lo informa y vuelve a pedirlo sin cerrarse.

2. Procesamiento:

- **Validación previa:** igual dimensión para suma y resta; columnas de A igual a filas de B para el producto; matriz cuadrada para determinante e inversa.
- **Operaciones básicas:** calcular el resultado mediante bucles anidados respetando las leyes del álgebra matricial.
- **Determinante:** expansión por cofactores, válida para cualquier orden n (en 3×3 mostrar también Sarrus), y reducción a forma triangular con operaciones de fila, llevando la cuenta de los intercambios y de los factores. Los métodos deben coincidir; la reducción es la eficiente, porque los cofactores requieren cerca de $n!$ multiplicaciones.
- **Inversa por Gauss-Jordan:** construir $[A|I]$ y reducirla por filas hasta $[I|A^{-1}]$. Si la matriz no llega a tener n pivotes, es singular y se detiene el proceso.
- **Inversa por matriz adjunta:** construir la matriz de cofactores, transponerla para obtener $\text{adj}(A)$ y aplicar $A^{-1} = (1/\det(A)) \cdot \text{adj}(A)$, solo si $\det(A) \neq 0$. Ambos métodos deben dar la misma inversa.
- **Comprobación automática:** tras obtener A^{-1} , el programa verifica que $A \cdot A^{-1} = I$ usando su propia función de producto (exacto con fractions, o contra una tolerancia si usan float).

3. Salida esperada:

- Mostrar las matrices de entrada impresas con un formato visual claro (filas y columnas ordenadas).
- Si las dimensiones no son compatibles o la matriz no es cuadrada, imprimir un mensaje explicativo de error (ej. "No se puede multiplicar: Columnas de A [3] ≠ Filas de B [2]").

- Si la operación es válida, mostrar la matriz resultante con sus respectivas dimensiones.
- En determinante e inversa: mostrar $\det(A)$, el proceso final $[I|A^{-1}]$ o $\text{adj}(A)$, y el diagnóstico: "La matriz es invertible: $\det(A) \neq 0$, tiene n posiciones pivote, sus columnas son L.I. y generan $\mathbb{R}^n$ " o "La matriz es singular (no tiene inversa): $\det(A) = 0$ ".
- **Opción 0 (Teoremas Clave):** además de lo definido en la Tarea 4, mostrar el teorema de la inversa de la Sesión 10 (a, b y c) y las caracterizaciones con asterisco del Teorema de la Matriz Invertible (c: n posiciones pivote, e: columnas L.I., h: columnas generan $\mathbb{R}^n$), junto con A invertible $\Leftrightarrow \det(A) \neq 0$ y $A^{-1} = (1/\det(A)) \cdot \text{adj}(A)$.

4. Verificador de propiedades (opción 9 del menú):

- Dadas dos matrices A y B cuadradas e invertibles del mismo orden n , calcular ambos miembros de cada igualdad, mostrarlos y concluir «Se cumple» o «No se cumple». Debe usar las mismas funciones del menú, sin repetir los cálculos. Propiedades de las Sesiones 10 y 11:
- Propiedad 1: $(A^{-1})^{-1} = A$
- Propiedad 2: $(AB)^{-1} = B^{-1}A^{-1}$
- Propiedad 3: $(A^T)^{-1} = (A^{-1})^T$
- Propiedad 4: $\det(A^{-1}) = 1/\det(A)$
- Propiedad 5 (operaciones de fila): al aplicar a A un intercambio de dos filas, un reemplazo ($F_i \rightarrow F_i + k \cdot F_j$) o la multiplicación de una fila por k , el determinante resulta $-\det(A)$, $\det(A)$ y $k \cdot \det(A)$, respectivamente. El usuario elige las filas y k .
- Propiedad 6 (matriz triangular): el determinante es el producto de las entradas de la diagonal. El programa reduce A a forma triangular y compara ese producto (corregido por los intercambios de fila y los factores usados) con la expansión por cofactores.

5. Casos de prueba sugeridos (para las capturas del informe):

- Producto válido: con $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ de 2×3 y $C = \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 3 \end{bmatrix}$ de 3×2 , el resultado esperado es $A \cdot C = \begin{bmatrix} 5 & 11 \\ 14 & 23 \end{bmatrix}$; además $C \cdot A$ es de orden 3×3 , por lo que $AC \neq CA$.
- Dimensiones incompatibles: con A de 2×3 y B de 2×3 , el programa debe mostrar «No se puede multiplicar: Columnas de A [3] $\neq$ Filas de B [2]».
- Matriz invertible: con $A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{bmatrix}$ se espera $\det(A) = 1$ y $A^{-1} = \begin{bmatrix} -24 & 18 & 5 \\ 20 & -15 & -4 \\ -5 & 4 & 1 \end{bmatrix}$ por ambos métodos. Con $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ se espera $\text{adj}(A) = \begin{bmatrix} 4 & -2 \\ -3 & 1 \end{bmatrix}$ y $A^{-1} = \begin{bmatrix} -2 & 1 \\ 3/2 & -1/2 \end{bmatrix}$.
- Matriz singular: con $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ se espera $\det(A) = 0$, solo 2 pivotes y el diagnóstico «La matriz es singular»; la inversa no se calcula.
- Verificador: con $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ y $B = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$ se espera $\det(A) = -2$, $A^{-1} = \begin{bmatrix} -2 & 1 \\ 3/2 & -1/2 \end{bmatrix}$, $\det(A^{-1}) = -1/2$, $(AB)^{-1} = \begin{bmatrix} 7/2 & -3/2 \\ -2 & 1 \end{bmatrix} = B^{-1}A^{-1}$ y $(A^T)^{-1} = \begin{bmatrix} -2 & 3/2 \\ 1 & -1/2 \end{bmatrix}$. Propiedad 5: intercambiar filas da $\det = 2$; $F_2 \rightarrow F_2 - 3F_1$ da $\det = -2$; multiplicar F_1 por 3 da $\det = -6$. Propiedad 6: $A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{bmatrix}$ se reduce con $F_3 \rightarrow F_3 - 5F_1$ y $F_3 \rightarrow F_3 + 4F_2$ a una triangular de diagonal 1, 1, 1, así que $\det(A) = 1$, igual que por cofactores.

4. ¿Cómo van a entregar la tarea?

Subirán un archivo comprimido .zip a la plataforma Moodle nombrado de la siguiente forma:

Programa 5_Grupo x_Apellido1_Apellido2.zip

Ese .zip debe traer dentro dos elementos:

1. El código Python: el proyecto Calculadora_Algebra_Lineal/ (main.py, modulos/ y teoremas/) con el Módulo 3 integrado, documentado según la Guía de documentación (sección 5).
2. El reporte en PDF: Nombrado Informe_Programa 5_Grupo x.pdf, que debe incluir:
 - Portada: Universidad, facultad, materia, docente, integrantes, carrera y grupo.
 - Explicación breve: De 1 a 2 páginas donde expliquen con sus palabras la lógica algorítmica del producto matricial, de la expansión por cofactores y de la reducción Gauss-Jordan sobre $[A|I]$.
 - Capturas de pantalla: Pruebas donde muestren la ejecución correcta de cada opción del menú, el manejo de un error de dimensiones incompatibles, una matriz invertible y una singular, y el verificador de propiedades.

5. Guía de documentación del código: comentar con criterio

La calculadora se construye durante todo el semestre y cada módulo será leído, probado y reutilizado por otras personas (compañeros, docente y ustedes mismos dentro de unas semanas). Por eso el código debe entenderse sin esfuerzo, pero **documentar no significa comentar todo**: un comentario en cada línea es tan difícil de leer como un código sin comentarios, porque el ruido oculta lo importante. El principio es: **el código dice qué hace; el comentario explica por qué lo hace así.**

Reglas de documentación (aplican a este programa y a todos los siguientes):

- **Encabezado del módulo:** al inicio de cada archivo .py, una docstring de 3 a 5 líneas que indique qué hace el módulo, qué tema de clase implementa y quiénes lo elaboraron.
- **Una docstring por función (1 a 3 líneas):** qué hace, qué recibe y qué devuelve. No se explica el código instrucción por instrucción.
- **Comentarios de línea solo para el porqué:** una decisión matemática o un caso especial (por qué se intercambian filas, por qué se compara contra una tolerancia, por qué se validan las dimensiones antes de operar). Si el comentario solo repite lo que ya dice el código (# suma 1 al contador), se elimina.
- **Nombres que se explican solos:** fila_pivote y num_variables_libres en lugar de x, aux o temp. Un buen nombre reemplaza muchos comentarios.

- **Funciones cortas y con una sola tarea:** como referencia, de unas 25 líneas o menos. Las funciones que calculan no usan `input()` ni `print()`; el menú y la impresión van en funciones separadas.
- **Qué no se hace:** comentar línea por línea, dejar bloques de código viejo comentados, escribir comentarios decorativos largos o pegar explicaciones que no puedan defender con sus propias palabras. El logotipo ASCII no cuenta como comentario: es parte de la salida del programa.

Señal de alerta (orientativa, no es una regla estricta): si más de una de cada cinco líneas de su archivo es un comentario, probablemente sobran; si una decisión matemática no trivial no tiene ningún comentario que explique su porqué, probablemente falta.

Ejemplos de comentarios que sí aportan en este programa (explican el porqué):

- `# Se intercambian filas si el pivote es 0: sin pivote no se puede eliminar la columna.`
- `# Sin n pivotes la matriz es singular: se detiene la reducción en lugar de dividir entre 0.`
- `# El signo (-1)**j sigue el patrón de cofactores: alterna según la posición de la columna.`
- `# Con float se compara contra una tolerancia (1e-9), no con ==, por el error de redondeo.`

Ejemplo: la misma función, dos formas de documentarla.

Sobre-comentado (evitar): el comentario repite el código y no aporta nada.

```
def producto(A, B):
    # creamos el resultado
    R = []
    # recorremos filas de A
    for i in range(len(A)):
        # creamos una fila nueva
        fila = []
        # recorremos columnas de B
        for j in range(len(B[0])):
            # variable suma
            s = 0
            # recorremos k
            for k in range(len(B)):
                # sumamos A[i][k] por B[k][j]
                s = s + A[i][k] * B[k][j]
            # agregamos s a la fila
            fila.append(s)
        # agregamos la fila a R
        R.append(fila)
    # devolvemos R
    return R
```

Documentado con criterio (lo esperado): una docstring y solo el comentario que explica el porqué.

```
def producto_matricial(A, B):
    """Devuelve A·B (matriz m x p). Requiere columnas de A == filas de B,
    condición que se valida antes de llamar a esta función.
    A: m filas de n elementos. B: n filas de p elementos."""
    m, n, p = len(A), len(B), len(B[0])
    resultado = []
    for i in range(m):
        fila = []
        for j in range(p):
            # Elemento (i, j) = fila i de A por columna j de B
            fila.append(sum(A[i][k] * B[k][j] for k in range(n)))
        resultado.append(fila)
    return resultado
```

En la defensa se preguntará la lógica de cada función: si pueden explicarla con sus palabras sin leer el comentario, la documentación cumple su propósito.

Antes de entregar, verifiquen:

- Cada archivo .py inicia con su docstring de módulo.
- Cada función tiene una docstring de 1 a 3 líneas.
- Los comentarios de línea explican el porqué, no el qué.
- No quedan bloques de código viejo comentados ni comentarios decorativos.
- Todos los integrantes pueden explicar cada función con sus propias palabras.

Esta guía se valora en el criterio «Calidad del código y documentación» de la rúbrica de la sección 6.